

Fine-Grained Dynamic Update Controller Synthesis: Component Changes and Requirement Lifetimes

ANONYMOUS AUTHOR(S)

Updating a controlled system can require component changes and safety requirements to take effect at different times. Fine-Grained Dynamic Update Controller Synthesis (FG-DUCS) formulates this problem through local transfers, individual requirement lifetimes and current-state initializers. From these contracts it constructs mixed-version behavior and synthesizes a policy between supplied controllers. We prove contract correspondence and a specialized lazy procedure's soundness and completeness for safe finite completion from every old entry under every outcome, preserving the selected new controller's continuations. Complete examples and merging obstructions establish why both forms of granularity matter. An MTSA implementation decides 26/27 adapted contracts within 1,200 s versus 24 with update-first priority, 17 with eager successors and 14 with full construction; 33/55 controlled comparisons exhibit fine-WIN/merged-LOSS. Guarantees cover declared lifetimes and quiescent commands/handover under explicit execution premises.

CCS Concepts: • **Software and its engineering** → **Software evolution**.

Additional Key Words and Phrases: dynamic software updating, discrete-event controller synthesis, supervisory control, on-the-fly synthesis

ACM Reference Format:

Anonymous Author(s). 2027. Fine-Grained Dynamic Update Controller Synthesis: Component Changes and Requirement Lifetimes. In *Proceedings of The ACM International Conference on the Foundations of Software Engineering (FSE 2027)*. ACM, New York, NY, USA, 21 pages.

1 Introduction

Verified endpoints leave their intervening update unspecified [3, 6, 13, 31]. A cell may need to move its workpiece before replacing an empty station. An audit requirement may need overlap while a role requirement forbids it. Updating therefore coordinates component changes and the requirements currently in force. It must safely finish from every old entry under every transformation outcome; safety alone permits infinite execution.

FG-DUCS formulates and solves updating with independently scheduled component changes and requirement lifetimes. Inputs specify local transfers, individual start/stop operations and current-state initializers. The construction generates mixed-version behavior; synthesis returns a policy, completion rank and compatible handover state, or proves impossibility. No complete update sequence is supplied. After handover the supplied new controller retains all its continuations.

DUCS uses global specification-stop/start and reconfiguration events [28, Defs. 4.1–4.3]; GR(1) updating switches between two specifications [2, Def. 3.1]. Both generate update models or bridges. Additional encodings can represent finer choices, but their native inputs lack the indexed lifecycle operations studied here. FG-DUCS defines their joint semantics and provides dedicated synthesis; Section 3.4 distinguishes this contribution from general expressive power.

C1: Fine-grained update problem. A contract and game construction for component transfers, individual requirement lifetimes and stateful activation, with correspondence and endpoint-preservation proofs (Section 3).

C2: Specialized synthesis. A sound and complete lazy procedure with independently checkable WIN/LOSS certificates (Section 4).

C3: Why granularity matters. Complete component and requirement examples, general merging obstructions, and an exact Rolling batch threshold (Sections 2 and 6.2).

C4: Implemented feasibility and cost. An MTSA tool evaluated on 27 adapted contracts and 55 controlled comparisons, including all adverse outcomes (Section 6).

Commands and handover require uncontrollable (UC) quiescence. Guarantees cover declared safety lifetimes and finite completion.

2 From a Complete Contract to a Completing Policy

Component granularity chooses what is replaced together; requirement granularity chooses which guarantees start or end together. Two contracts separate these choices.

2.1 Cell: Complete Local Input and Whole Policy

Consider a constructed two-station cell with exactly one workpiece [20, 28]. Both transfers require an empty station. A keeps its behavior; new B requires inspection before departure, while the old rule forbids inspection. Ordinary events move or inspect the workpiece. New-start before old-stop prevents an unprotected gap; a pending inspection waits for old-stop. Inspection can repeat safely forever.

Table 1 specifies every component, endpoint and monitor. The interval requires exactly one workpiece, which plant transitions preserve. All labels are controllable; new-start before old-stop is the only dependency. Transfers are $g_A = g_B = \{(e, e)\}$, where e means empty.

Table 1. Complete Cell components, controllers and monitors. o/n denote old/new versions; h/e mean holding/empty. Parentheses mark initial states. m_A, m_B move to A, B ; j inspects at B .

Object	States (initial)	Ordinary transitions
A^o, A^n	$h, e (h)$	$h \xrightarrow{m_B} e, e \xrightarrow{m_A} h$
B^o	$h, e (e)$	$e \xrightarrow{m_B} h, h \xrightarrow{m_A} e$
B^n	$h, e (e)$	$e \xrightarrow{m_B} h, h \xrightarrow{m_A} e, h \xrightarrow{j} h$
C^o	$c_0 (c_0)$	$c_0 \xrightarrow{m_A, m_B} c_0$
C^n	$c_0, c_1 (c_0)$	$c_0 \xrightarrow{m_A, j} c_0, c_0 \xrightarrow{m_B} c_1, c_1 \xrightarrow{j} c_0, c_1 \xrightarrow{m_B} c_1$
t_{rone}	$A, B, \text{err} (A)$	$A \xrightarrow{m_B} B, B \xrightarrow{m_A} A, A \xrightarrow{m_A} \text{err}, B \xrightarrow{m_B} \text{err}$
t_{rld}	$k, \text{err} (k)$	$k \xrightarrow{j} \text{err}$
t_{rinspect}	$c, d, \text{err} (c)$	$c \xrightarrow{m_B} d, d \xrightarrow{j} c, d \xrightarrow{m_A} \text{err}$

k, c, d mean old-safe, inspection-clear and inspection-pending; c_0, c_1 are distinct controller states.

Component/controller alphabets are exactly their displayed labels; unlisted edges are absent. Unlisted non-error monitor edges self-loop, and err absorbs every label. Each comma-separated label denotes an edge.

Endpoint tuples list A, B , controller and its requirement monitor. The old initial tuple is $x_A = (h, e, c_0, k)$ and its other reachable tuple is $x_B = (e, h, c_0, k)$; both are entries. The new initial tuple is $z_A = (h, e, c_0, c)$; its other reachable tuples are $y_B = (e, h, c_1, d)$ and $z_B = (e, h, c_0, c)$. Only $\{z_A, z_B\}$ are loadable targets: inspection must be clear at handover. The interval initializer records the holder in either all-old tuple. The new initializer admits all eight version-tagged one-workpiece tuples, installing d if B holds and c otherwise. These data specify the contract without an update sequence.

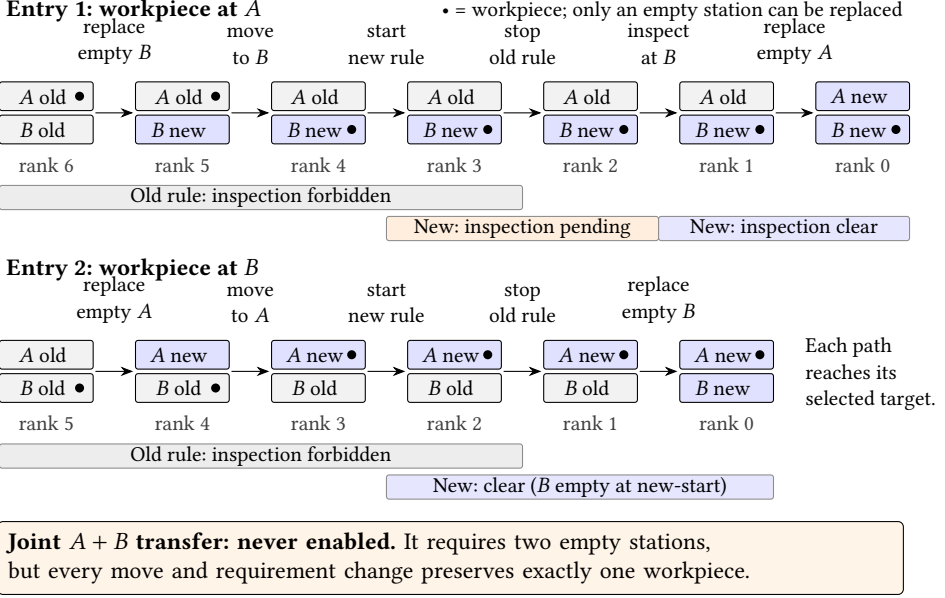


Fig. 1. A complete local policy from both old entries. Bands mark active rules and new-rule inspection memory; new-start precedes old-stop. Each arrow has one action and one outcome, decreasing rank toward its selected new target. Joint transfer merges these local commands with endpoints and requirements unchanged; this is not a DUCS encoding.

Feasibility and completion. Moving the workpiece enables local replacements; joint replacement requires both stations empty, impossible under any move or boundary. Endpoints and requirements remain fixed. Inspection may repeat forever safely, but every policy arrow decreases rank from six or five to zero, proving all-outcome handover from both entries.

A whole policy from the local input. Figure 1 covers both entries and targets: 13 states and 11 edges. Shown events have singleton outcomes; other controllables are disabled. Activation on the first path creates an inspection duty cleared before handover to z_B ; the second reaches z_A . Technical Appendix B and S1 give paths and closure.

2.2 Policy: Scheduling Requirement Lifetimes

Policy(2) requires audit overlap and role nonoverlap. Table 2 gives its complete input. Write o, n for old/new, u, l for unlogged/logged, A, R for audit/role, and ℓ_z, a_z for version- z log/access.

From q_u , the saved policy executes start_A , stop_A , stop_R , ρ , start_R , reaching z_u . From q_l , service first reaches q_u . Ranks five/six decrease on every edge; other controllables are disabled. Technical Appendix I depicts this whole policy.

Both merged entries lose: the first global start overlaps roles; the first global stop removes audit coverage. Ordinary events and transfer preserve these bits, so every completing path encounters error. The saved checks confirm closure and both member orders. This compares boundary granularity, not DUCS expressiveness. Service continuity is not required.

Table 2. Complete Policy(2) input; all events controllable. Unlisted plant/controller edges are absent; unlisted monitor edges self-loop, including at error err. Monitors have alphabet Σ .

Part	Finite definition
Plant/transfer	Each version has sole initial state r with loops on $\Sigma_N = \{\ell_o, \ell_n, \text{service}, a_o, a_n\}$. Transfer ρ changes its version with $g = \{(r, r)\}$.
Controllers	C^z ($z \in \{o, n\}$) has alphabet Σ_N , initial u , edges $u \xrightarrow{\ell_z} l \xrightarrow{\text{service}} u$, and a_z loops at u, l . An unreachable third state has all Σ_N loops.
Endpoint monitors	A_z : initial u , states u, l, err , edges $u \xrightarrow{\ell_z} l \xrightarrow{\text{service}} u$ and $u \xrightarrow{\text{service}} \text{err}$. R_z : initial ok, states ok, err, edge $\text{ok} \xrightarrow{a_z} \text{err}$, where $\bar{z}$ is the other version.
Interval monitor	Initial 1010; states $\{b \in \{0, 1\}^4 : (b_1 \vee b_2) \wedge \neg(b_3 \wedge b_4)\} \cup \{\text{err}\}$. Bits track active (A_o, A_n, R_o, R_n). Respective stop/start commands write $b_1 = 0, b_2 = 1, b_3 = 0, b_4 = 1$; unsafe results enter err. Other events preserve bits.
Initialization	Four boundaries and ρ , once each; no precedence. Old monitors are A_o, R_o , new are A_n, R_n . On either (o, r) or (n, r) , initialize $A_n = u, R_n = \text{ok}$; initialize the interval to 1010 only at (o, r) . Domains are complete.
Entries/targets	For $x \in \{u, l\}$, entry q_x admits old tuple $((o, r), C^o = x, A_o = x, R_o = \text{ok})$; loadable targets are $z_x = ((n, r), C^n = x, A_n = x, R_n = \text{ok})$. Admission, pending commands, activation and goal tests follow Section 3.
Boundary merge	One global start activates both new monitors; one global stop deactivates both old monitors. Compose member observations audit then role; other inputs stay fixed.

3 Defining an Update Contract

Seven supplied groups determine the game: *event control*, *fixed old/new controllers*, *components and local transfers*, *old/new/interval requirements*, *activation initializers*, *optional command precedence*, and *handover targets*. They do not prescribe a sequence. Section 5.1 connects initialized monitor states to declared requirement scopes.

3.1 Supplied Choices

Finite LTSs and safety monitors. A labeled transition system (LTS) $\mathcal{L} = (S, \Sigma_{\mathcal{L}}, \delta, s^0)$ has nondeterministic transitions $\delta : S \times \Sigma_{\mathcal{L}} \rightarrow 2^S$. Partition events as $\Sigma = \Sigma_{\text{uc}} \uplus \Sigma_{\text{c}}$: controllers may disable controllables but cannot select outcomes or suppress uncontrollable (UC) events. A synchronous-product event requires a transition in every component whose alphabet contains it; other components retain their state [21]. A monitor is an automaton that checks a requirement as events occur. Requirement r has a total deterministic monitor $t_r = (M_r, \Sigma, \eta_r, m_r^0, F_r)$ with absorbing errors for violating prefixes [1, 19]. The empty monitor product has one non-error state. Monitors observe labels; policies also observe successor component states.

Components and transfers. For each $i \in I$, let $e_i^{\text{old}}, e_i^{\text{new}}$ be the pre-update and post-update components and let $g_i \subseteq S_{e_i^{\text{old}}} \times S_{e_i^{\text{new}}}$ be their state-transfer relation. Define $g_i(s) = \{s' \mid (s, s') \in g_i\}$. The supplied g_i includes every modeled transformation outcome. Let $E^v = \prod_{i \in I} e_i^v$.

Requirements and activation. Let pairwise-disjoint $R^{\text{old}}, R^{\text{new}}, R^{\text{upd}}$ contain the pre-update, post-update, and update-interval safety requirements, respectively. For each $r \in R^{\text{new}} \cup R^{\text{upd}}$, the input supplies a partial *monitor-initialization map* ι_r from version-tagged component tuples to $M_r \setminus F_r$. Its domain specifies where activation is allowed; in the cell, initialization records whether inspection is pending. For history-based requirements, residual soundness (RS) requires every continuation

accepted by the installed state to be safe after every declared activation history. Activation-scoped invariants instead retain current facts and constrain behavior from their start; both interpretations are defined in Section 5.1. For $r \in R^{\text{upd}}$, $\text{dom } t_r$ includes every reachable old component state.

Endpoints and handover targets. For $v \in \{\text{old}, \text{new}\}$, the supplied finite controller C^v preserves uncontrollables and makes $E^v \parallel C^v$ deadlock-free and satisfy R^v . Hiding controller auxiliary events leaves observation alphabet $\bigcup_i \Sigma_{e_i^v}$; the union over both versions is the ordinary alphabet Σ_N . Write Reach for initial-state reachability and

$$\text{CL}^v = E^v \parallel C^v \parallel \parallel_{r \in R^v} t_r.$$

Monitors are passive: every endpoint transition originates in $E^v \parallel C^v$; a monitor only updates on that label and adds no enabled event. The input supplies nonempty, checked targets $Z_{\text{load}} \subseteq \text{Reach}(\text{CL}^{\text{new}})$ with component, controller and new-monitor states. FG-DUCS keeps the supplied new controller fixed. The nine adapted benchmark inputs admit all of $\text{Reach}(\text{CL}^{\text{new}})$; constructed contracts may select a proper subset, as Cell does. For a supplied maximally permissive controller this is its reachable winning region, equal to the full region only if all winning states are represented and reachable.

Update commands and order. The set of update events, each executed exactly once, is

$$\Sigma_{\text{upd}} = \{\rho_i \mid i \in I\} \cup \{\text{stop}(r) \mid r \in R^{\text{old}}\} \cup \{\text{start}(r) \mid r \in R^{\text{new}}\}. \quad (1)$$

These controllable events transfer components, stop old monitors or start new ones, respectively, and are disjoint from ordinary events Σ_N : $\Sigma = \Sigma_N \uplus \Sigma_{\text{upd}}$ and $\Sigma_{\text{upd}} \subseteq \Sigma_C$. Each transfer event ρ_i is one atomic model transition: the controller chooses the update, but cannot choose its g_i outcome. They require uncontrollable quiescence: no modeled uncontrollable event may be enabled when an update command is issued.

An optional strict partial order $< \subseteq \Sigma_{\text{upd}} \times \Sigma_{\text{upd}}$ expresses precedence among update operations. For a pending-event set P , event a is order-enabled exactly when $a \in P$ and every $a' < a$ has left P . Monitors self-loop on update events that they do not specify.

Eligible contracts. Eligibility requires finite, consistently typed states/events and controllability; total deterministic monitors with absorbing errors; typed transfers and non-error initializers covering interval entry; and strict partial-order dependencies. Endpoints are safe, deadlock-free and uncontrollable-preserving, with controller alphabets covering environment labels; targets are nonempty and reachable. Justifying initializers against their declared requirement meaning is separate from game eligibility. Interval and post-handover duties belong to R^{upd} and R^{new} ; $\text{start}(r^{\text{new}}) < \text{stop}(r^{\text{old}})$ enforces overlap.

3.2 States, Transitions, and Completion

States and all entry states. An update state $q = (\xi, \mu, P)$ records each component's version/state $\xi(i) = (v, s)$ with $s \in S_{e_i^v}$, each monitor's state $\mu(t_r) \in M_r \cup \{\perp\}$ ($\perp$ means inactive), and pending commands $P \subseteq \Sigma_{\text{upd}}$. Here $\rho_i \in P$ exactly when component i is old. Old monitors remain active until their stop leaves P ; new monitors become active when their start leaves P ; update monitors are always active.

Admission $\text{Init}(z)$ copies z 's component and old-monitor states. It initializes interval monitors through $\iota_r(\xi)$, leaves new monitors inactive and sets $P = \Sigma_{\text{upd}}$. The policy replaces C^{old} , discarding its state and scheduling restrictions; undeclared objectives, including old liveness, are not preserved. Old monitors remain until stopped. The entry set is

$$Q_0 = \{\text{Init}(z) \mid z \in \text{Reach}(\text{CL}^{\text{old}})\}. \quad (2)$$

All-entry coverage avoids waiting for a favorable old state; an admission predicate would define a different contract. In the cell, $Q_0 = \{q_A^0, q_B^0\}$ distinguishes the two holders.

Four kinds of induced transitions. For $\xi(i) = (v, s)$, component i uses e_i^v . The active-component product determines enabled ordinary uncontrollables $U(q)$, including those with unsafe outcomes, independently of updates and goals. For every $a \in \Sigma_{\text{upd}}$, impose

$$U(q) \neq \emptyset \implies \text{Post}(q, a) = \emptyset. \quad (3)$$

This global *uncontrollable quiescence* differs from transaction quiescence [18] and tranquility [35]. It is a command restriction, not a consequence of local transfer. Ordinary controllables may coexist with uncontrollables; the UC-priority lemma later justifies suppressing them. $\text{Post}(q, a)$ contains exactly the following outcomes; unmentioned coordinates remain unchanged. “Advance” means $\mu'(t_r) = \eta_r(\mu(t_r), a)$ for the specified active monitors.

Ordinary event $a \in \Sigma_N$. If a belongs to the local event set of at least one active component and every active component having a can transition, take every synchronous-product outcome and advance every active monitor.

Component update ρ_i . If $U(q) = \emptyset$, order-enabled, and $\xi(i) = (\text{old}, s)$, set $\xi'(i) = (\text{new}, s')$ for each $s' \in g_i(s)$, delete ρ_i from P , and advance every active monitor.

Requirement deactivation $\text{stop}(r)$. If $U(q) = \emptyset$ and order-enabled, advance every active monitor except t_r , retain inactive monitors at $\perp$, set $\mu'(t_r) = \perp$, and delete $\text{stop}(r)$ from P ; t_r does not observe it.

Requirement activation $\text{start}(r)$. If $U(q) = \emptyset$, order-enabled, and $\xi \in \text{dom } \iota_r$, advance every active monitor, retain other inactive monitors at $\perp$, install $\mu'(t_r) = \iota_r(\xi)$, and delete $\text{start}(r)$ from P ; t_r begins online observation afterward.

On the cell path starting with A holding, start preserves old/empty A and new/holding B , installs inspection-pending in the new monitor and removes itself from P . The old and interval monitors stay active.

If a precondition is false, the successor set is empty; every nondeterministic outcome is included and environment-chosen. Let $\mathcal{Q}$ contain every state reachable from Q_0 through these transitions. *Safety and handover-ready goals.* An update state $q = (\xi, \mu, P)$ is safe, written $\text{Safe}(q)$, exactly when every active monitor is outside its error states. For $P = \emptyset$ and $z \in Z_{\text{load}}$, write $q \sim z$ when component states (without version tags) and all new-monitor states agree. The handover map κ selects such a full endpoint state z . Goals are

$$Q_{\text{goal}} = \{q = (\xi, \mu, \emptyset) \in \mathcal{Q} \mid \text{Safe}(q) \wedge U(q) = \emptyset \wedge \exists z \in Z_{\text{load}} : q \sim z\}. \quad (4)$$

At a goal, all components and endpoint monitors are new; interval obligations end and matching new monitors continue. Handover disables ordinary controllables and installs $\kappa(q)$'s controller state, outside Σ and the rank. No uncontrollable can compete. A matching UC-enabled state is not a goal; compatible targets may differ in later safe behavior.

State-dependent policies and finite completion. A memoryless update policy is a function $\sigma : \mathcal{Q} \setminus Q_{\text{goal}} \rightarrow 2^{\Sigma_c}$ selecting only events with nonempty successor sets.

The possible next states under this policy are

$$\text{Succ}_\sigma(q) = \bigcup_{u \in U(q)} \text{Post}(q, u) \cup \bigcup_{a \in \sigma(q)} \text{Post}(q, a). \quad (5)$$

The environment chooses any uncontrollable and any permitted-event outcome. An uncontrollable self-loop blocks updates by Equation (3) and permits a losing infinite run. The returned policy retains one controllable at a quiescent state.

Let $\text{Reach}_\sigma(Q_0)$ be the policy closure from Q_0 , stopping at goals.

Runs, winning, and witnesses. A run follows the policy successors and stops at a goal. It is *maximal* if infinite or ending at a goal or a state without successors. A single policy σ is winning for an FG-DUCS instance if, from every $q_0 \in Q_0$, (i) every reachable update state is safe, (ii) every reachable non-goal state has a successor, and (iii) every maximal update run reaches Q_{goal} after finitely many events. This *finite completion* counts observed transitions and assumes no fairness.

A *progress-ranking function* for a winning policy is $\text{rk} : \text{Reach}_\sigma(Q_0) \rightarrow \mathbb{N}$, with value 0 at goal states and satisfying $\text{rk}(q') < \text{rk}(q)$ for every non-goal state q and every $q' \in \text{Succ}_\sigma(q)$.

The FG-DUCS problem. The seven supplied groups define the eligible game. Decide whether one policy safely completes it from every entry under every permitted event and outcome. Return $(\sigma, \text{rk}, \kappa)$ covering all Q_0 , or a losing-region certificate.

3.3 Why the Generated State Represents the Contract

A *local update history* $\ell = z(a_1, o_1) \cdots (a_n, o_n)$ begins at an old closed-loop tuple and records each event and its realized local successors. Components follow their active-version transitions and local transfers; boundaries preserve physical states. Monitor memories fold their active intervals: stop excludes its own boundary; start installs its initializer after the boundary. Initially inherit old memories, initialize interval monitors and release old control, leaving new monitors inactive.

Ordinary events retain all combinations of participating local outcomes. Commands require UC quiescence, unexecuted status, completed predecessors and a transfer/initializer value where applicable. Quiescence includes unsafe UC outcomes. A source policy selects controllable labels and must retain every UC and every permitted outcome. Every maximal history must safely reach a quiescent matching target after all commands. Technical Appendix A gives the full recurrences.

PROPOSITION 3.1 (ADEQUACY OF THE GENERATED UPDATE GAME). *For an eligible contract, map each local update history to its current component tuple, active monitor memories and unexecuted commands. Histories with the same image admit the same next events and all the same local outcomes, agree on safety, and have the same handover targets. This map gives a label- and outcome-preserving correspondence between local update histories and paths of the generated game. A history-dependent policy satisfying the source obligations exists exactly when a winning policy exists in that game.*

PROOF. Let $F(h)$ be the tuple obtained by folding the stated source operations over history h , with pending commands the complement of executed commands. At entry, $F(h) = \text{Init}(z)$, giving exactly Q_0 . Suppose a history ends at (ξ, μ, P) . For an ordinary event, both descriptions require the same participating component edges, retain their full Cartesian product and advance every active monitor. For a transfer, both require quiescence, pending status, completed predecessors and an outcome of the same g_i ; they change only component i , advance active monitors and consume that command. A stop removes its own monitor without consuming the boundary and advances the others. A start advances existing monitors and installs the same initializer, without a second observation. Thus each permitted label and each outcome has exactly the same successor image in both descriptions; no choice of outcome is introduced or removed. Quiescence counts unsafe UC outcomes in both. Safety and compatible targets depend on identical coordinates.

Induction gives a game path for every source history. Conversely choose an old tuple witnessing the first game state and lift each edge by the one-step property, retaining its outcome. The same recurrence lifts infinite paths. Histories with the same image admit identical future extensions because old control is released at entry. A game policy therefore lifts through F . Conversely, fix one old witness for each entry and consistently lift each game path from it; applying a source policy to that lifted history defines a game history-dependent policy. Both translations preserve every enabled UC event, every selected-label outcome, unsafe prefixes, deadlocks and infinite avoiding runs. Winning existence is therefore equivalent for history-dependent policies. Theorem 4.1’s losing-closure argument rules out every history-dependent winner on failure and returns a state-based winner on success, establishing the stated equivalence. $\square$

This proves correspondence for the supplied contract; requirement meaning and Java frontend correctness remain separate obligations. Technical Appendix A gives the full source recurrences and expanded proof. The construction reconnects each goal to the supplied endpoint.

PROPOSITION 3.2 (PRESERVATION OF THE SELECTED ENDPOINT). *For a returned WIN certificate and reached goal q , the new-phase submodel reachable after handover at q is label-preserving isomorphic to CL^{new} rooted at $\kappa(q)$. Hence their finite and infinite ordinary-event traces coincide.*

PROOF. Handover retains matching components and monitors and installs the selected controller state. Each new-phase state copies an endpoint state with exactly its labeled transitions, without further policy restriction or return to updating. This bijection preserves and reflects edges, hence finite and infinite paths. $\square$

Continuation properties established at the selected state are preserved; target selection can change which continuation applies. Execution additionally requires the adapter premises in Section 5.

Why matching and reconnection both matter. The Cell prefix $\rho_B, m_B, \text{start}, \text{stop}, \rho_A$ safely executes every command but leaves inspection pending. Matching physical states alone would allow loading clear target z_B , whose immediate m_A violates that active duty. Matching new-monitor memory excludes this reset. Conversely, a new closed loop with controllable self-loops a, b and goal *true* stays legal, deadlock-free and safe if b is disabled [28, Defs. 3.9–3.10, 4.3], but loses its b -continuations. Reconnection retains them. Matching protects the continuing obligation; reconnection preserves the selected controller’s behavior.

With fixed local data, equal entry sets and projected target sets permit WIN/LOSS certificate reuse across endpoint controllers (Technical Appendix O).

3.4 What the Local Interface Generates

Table 3 distinguishes the supplied structures from the generated result. DUCS already constructs an update environment and goal; GR(1) updating already constructs a bridge. From the supplied local contract, FG-DUCS constructs mixed-component transitions, implements individual monitor lifetimes, and derives handover target tests.

Concrete Cell constructions. Our DUCS witness supplies eight mixed stages, three endpoint states, empty-station guards, inspection/lifetime fluents and a target gate; DUCS generates its protocol. The GR(1) witness supplies version/lifetime variables, $p' = [h = B]$ at start, release at stop and a target test; its algorithm generates switching bookkeeping. Both admit the Cell paths; the appendices distinguish their endpoint guarantees. Technical Appendices K and P give the full analytical constructions, including the DUCS eventual-handover condition beyond native safety- T [28, Sec. 5.3]. These are neither solver runs nor general translations or encoding-size lower bounds.

Table 3. Input and generation responsibilities. Existing guarantees are retained in the comparison; the rows do not claim a general translation or measured modeling-effort reduction.

Interface	Supplied structure	Generated result and guarantee
DUCS [28, 4.1–4.3, 5.1]	Environments/goals, old controller, global transfer, transition requirements and update fluents	Update environment/goal, update-and-subsequent controller and controller-transfer relation; legal, deadlock-free completion from every old-controller state
GR(1) update [2, Def. 3.1, Secs. 4–4.1]	Old/new specifications and switching condition	New controller, bridge and switching book-keeping; bounded switching and new-specification satisfaction from winning states
FG-DUCS	Local components/transfers, requirement lifetimes/initializers and fixed endpoints	Mixed game and WIN/LOSS certificates; local-history correspondence and selected-endpoint continuation

4 Synthesizing and Checking an Update Policy

Lazy explores products on demand using finite reachability reasoning [7, 8, 34].

A *bucket* is $\text{Post}(q, a)$; an empty bucket disables a . At a UC-enabled state, disabling controls preserves winning because every UC outcome already requires handling (Lemma A.1).

4.1 Lazy Successor Generation

Algorithm 1 constructs a reachability attractor [5]. It tracks discovered states V , states $X \subseteq V$ with all UC candidates queried, and proved winners W . States in X may retain unqueried controls; W may omit winners.

Expansion queries every UC candidate. If none is enabled, it queries controls through the next nonempty bucket or exhaustion; Retry resumes unfinished cursors. Each query retains all outcomes. Candidates include local events and precedence-enabled commands in any fixed finite order.

Promote. A safe expanded state wins when its nonempty proving set—all UC successors, or one control bucket at quiescence—lies in W ; its rank is one above their maximum. Reverse incidences trigger predecessor reconsideration. Failure requires exhausted candidates and a promotion fixed point. Once all entries win, extraction selects one lower-rank control bucket at each reached quiescent non-goal, none elsewhere; it need not follow promotion or minimize length.

4.2 Correctness and Certificates

THEOREM 4.1 (TERMINATION, SOUNDNESS, AND COMPLETENESS). *For every eligible finite FG-DUCS instance with Equation (3) and quiescent goals in Equation (4), Algorithm 1 terminates. A returned $(\sigma, \text{rk}, \kappa)$ keeps every active monitor non-error, preserves every uncontrollable event and every outcome of each policy event, reaches Q_{goal} finitely without deadlock, and gives each reached goal a compatible target in Z_{load} for the supplied game. It returns no solution iff no winning policy covers Q_0 in that supplied game.*

PROOF. *Coverage.* Initialization/discovery queues every new safe non-goal in Open. Only expansion removes it, after querying all UC candidates. At quiescence, expansion/retries advance a finite control cursor and retain every nonwinner with candidates left in Retry; promotion removes only winners. Thus all expanded states have complete UC outcomes, unfinished quiescent nonwinners remain retryable, and unexpanded safe non-goals remain open.

Algorithm 1: Lazy**Input:** Typed update contract (Section 3)**Output:** $(\sigma, \text{rk}, \kappa)$ or (no solution, L)

```

1  $V \leftarrow Q_0$ ;  $X, \text{Retry} \leftarrow \emptyset$ ;  $W \leftarrow V \cap Q_{\text{goal}}$ ;
2  $\text{Open} \leftarrow \{q \in Q_0 \mid \text{Safe}(q) \wedge q \notin Q_{\text{goal}}\}$ ; set  $\text{rk}(q) \leftarrow 0$  for  $q \in W$ ;
3 while true do
4   Add queued goals to  $W$  once with rank zero; exhaust Promote, retaining witnesses/ranks, removing winners
   from Retry and propagating predecessors;
5   if  $Q_0 \subseteq W$  then return  $(\sigma, \text{rk}|_{\text{Reach}_\sigma(Q_0)}, \kappa)$  constructed from the goal witnesses and retained lower-rank
   choices;
6   if  $\text{Open} \neq \emptyset$  then
7     Remove  $q$  from Open, add it to  $X$ , and query all uncontrollable candidates;
8     Close the control cursor if UC-enabled; otherwise initialize it and query to the next nonempty bucket or
     exhaustion;
9     Insert  $q$  into Retry iff  $q \notin W$  and its cursor has remaining candidates;
10  else
11    if  $\text{Retry} = \emptyset$  then return (no solution,  $L \leftarrow V \setminus W$ );
12    Remove  $q$  from Retry; query to the next nonempty bucket or exhaustion; reinsert iff  $q \notin W$  and its
    cursor has remaining candidates;
13  Add every queried successor to  $V$ ; queue newly discovered goals for rank zero and safe non-goals for Open
    (unsafe states stay outside  $W$  and Open);

```

Termination. Each state is expanded/promoted once and each candidate queried once. Reverse incidences are processed on promotion, so the finite search terminates.

Soundness. Goals start at rank zero with compatible handover states. By induction, each promotion adds a safe state above a complete nonempty proving set of previously ranked winners: every UC successor, or every outcome of one control event. Queried buckets are immutable. Extraction retains all UC events, chooses one such control bucket at quiescence, and disables other controls. Every reached non-goal is safe, has a successor, and every successor has strictly smaller natural-number rank. Consequently no maximal run is an infinite run or a non-goal deadlock, and every run reaches a matching quiescent goal. The monitor rules advance exactly the memories active over each event; induction from entry establishes every declared safety interval. Handover retains the matching new memories.

Completeness. At failure both exploration queues are empty and promotion has reached a fixed point. Some entry is in $L = V \setminus W$, and L contains no goal. Coverage ensures that each safe state in L was expanded. If it is UC-enabled, at least one UC outcome lies in L , since otherwise it would be promoted. At quiescence every control candidate is exhausted; each enabled control bucket has an outcome in L for the same reason. Against any history-dependent policy the environment can select such an outcome after every history. Disabling all events deadlocks; otherwise the resulting run reaches an unsafe state or stays outside the goals forever. Hence no winning policy covers that entry. A winning policy therefore rules out failure and, by termination, forces success. Finally, UC priority loses no winning policy: removing optional controls at a UC-enabled state removes runs without introducing a deadlock, and the restricted policy remains admissible in the original game. $\square$

Checking the returned evidence. For WIN, let $D_\sigma = \text{Reach}_\sigma(Q_0)$. The checker requires exactly D_σ as the rank domain, safety throughout it and, at every non-goal, enabled selected controls, nonempty successor sets inside it, and strict rank decrease for every outcome. At goals it checks

rank zero, UC quiescence and compatible $\kappa(q) \in Z_{\text{load}}$, with $\text{dom } \kappa = D_\sigma \cap Q_{\text{goal}}$; omitted policy entries are empty.

For LOSS, $L \subseteq Q$ must include an entry state and exclude all goals. Every safe $q \in L$ must have some UC outcome in L when UC is enabled; otherwise every enabled controllable must have an outcome in L . Unsafe states and non-goal deadlocks lose. These conditions certify search relative to shared Post and endpoint predicates [17]; the implementation and independent checks in the main paper state the trust boundary (Section 6).

4.3 Exploring the Compiled Contract on Demand

With indexed predicates and constant-time bookkeeping, search is linear in discovered states, queried candidates and returned outcomes, excluding preprocessing and Java sorting/scans. Lazy may explore the whole reachable graph. For independent one-state updates, transfer-first Lazy instead visits $n + 1$ states versus Direct-Full's 2^n subsets; Technical Appendices E and J give the accounting and complete default-order proof.

5 Interpreting and Executing a Winning Policy

5.1 Giving Requirements Their Intended Lifetimes

An initializer connects monitor memory to an obligation at activation. Cell's new duty covers the workpiece already at B , so its initializer records occupancy (Table 1; Figure 1).

An exact activation obligation with ordinary work. For Cell, define an owed-inspection bit at activation to be 1 exactly when B holds the workpiece. Thereafter m_B sets it, j clears it, and m_A is forbidden while it is 1; other labels preserve it. Encoding 0 by c and 1 by d makes every non-error monitor transition identical to this obligation's rule, and precisely the forbidden departure enters error. Initialization and induction on the active suffix therefore establish exact enforcement. In Figure 1, the first path starts with bit 1 and ordinary inspection clears it; the second starts with bit 0. This proof covers Cell's declared duty and both entries.

Obligations that begin at activation. ACT starts a pure invariant with actual post-boundary observer values; it checks activation and the subsequent active suffix without counting earlier violations. Exact continuation language from a safe reference history with the same observer vector suffices (Proposition M.1, Technical Appendix M). Sections 6.2.1 and 6.3 apply this condition to source-derived examples.

When prior histories are part of the requirement. Let S_r be requirement r 's prefix-closed safe language, $H_r(\xi)$ its declared activation histories at observation ξ , and $L_r(m)$ the safe continuations accepted from monitor state m . Residual soundness (RS) requires $L_r(\iota_r(\xi)) \subseteq \bigcap_{h \in H_r(\xi)} \{w : hw \in S_r\}$. New-requirement histories include start once; initialized monitors consume later events. Equality is exact; strict inclusion may reject feasible updates. Empty $H_r(\xi)$ makes the right side universal. Synthesis establishes neither history-scope completeness nor intent (Technical Appendix B.1).

A Boolean event-history variable, or *fluent*, records setting/resetting labels. The evaluation distinguishes three history scopes:

A (activation-safe), for new requirements (NEW): admit only safe pre-activation histories consistent with the observed component state.

E (entry-scoped), for interval requirements (UPD): start the reference at update entry.

H (history-inclusive): count pre-activation violations too; not claimed. Synthesis does not establish whether the chosen scope matches intent (S3). Cell shows why scope matters: clear is exact for a reference starting without inspection debt, whereas its occupancy-based duty needs pending

when B holds the workpiece. A inclusion alone establishes neither that duty nor ACT; activation arguments use the declared source meanings.

5.2 Execution and Handover

The execution connection has four distinct premises (Technical Appendix H):

A1: endpoint observations preserve component/monitor states and same-labeled endpoint edges.

A2: update control disables only unselected controllables, preserves every UC event/outcome, reports the successor and eventually observes a next event.

A3: commands start quiescently and finish atomically, finitely and failure-free with an observed modeled successor.

A4: entry observes/copies the actual old state; quiescent handover installs the selected compatible state, using finite failure-free linearizable boundaries [15].

Under A1–A4, every execution maps to the linked model and reaches handover within the entry rank (Theorem H.1). Complete activation histories and RS additionally justify history-based interpretations. No event-choice fairness is assumed. Internal divergence and unobservable outcomes are excluded; rank bounds events, not time. Table 5 separates these premises from the scope checks and proofs actually obtained.

6 Implementation and Evaluation

6.1 Design and Implementation

Four FG procedures use 27 contracts from 9 inputs. Base has no interval requirement; R1 restricts ordinary actions and R2 orders boundaries versus transfers. PC2-Rolling adds continuous availability. Legacy, our DUCS/GR fork, synthesizes subsequent control instead.

Implementation. Our Modal Transition System Analyser (MTSA) extension [9] checks certificates [25]; Link checks old–update–new composition, entries and targets. An independent oracle parses restricted Finite State Processes (FSP). Finite-JSON runs save losing membership; FSP benchmark runs report only its statistics. Runtime conformance is unchecked.

Use of generative AI. Generative AI assisted design, implementation, tests, experiments, analysis, proofs and writing; Claude Opus 5.5 and ChatGPT Pro/Latest provided editorial reviews. Authors checked outputs through tests, certificates, oracles and primary sources and retain responsibility. Correlated faults remain possible (Technical Appendix N); no human evaluation was performed.

Constructing the benchmark inputs. Nine inputs—GSM, Industry, MetaSocket, PC1, PC2, PowerPlant, Railcab, Surveillance and Workflow—adapt eight DUCS sources; PC1/PC2 share a parent. Base/R1/R2 vary only interval requirements. Adaptations make PC tool results UC, shorten Railcab transfer paths while preserving both outcomes, and omit PowerPlant’s initial ENV transfer pair. They also change safety and ordering; whole-problem equivalence is unestablished (Technical Appendix J).

Users supply FSP goals and component/transfer declarations. MTSA’s GR backend fixes endpoints; all reachable new-endpoint states are targets. The frontend compiles monitors, enumerates physical closure and fills new-initializer tables from current fluent observers; interval tables use post-entry values. These generated maps use global tuples. Observers track nonlocal labels and survive commands/transfers. Lazy generates the remaining monitor/command product on demand. JVM cost includes preprocessing.

Inputs have 1–5 components, 4–131 entries and 3–26 commands (Technical Appendix C). Only Railcab transfers branch; none declares precedence. R2 encodes order in monitors. GSM/PC1’s single transfer makes merging the identity.

Exploration variants. All methods share entries, successors, goals, checks and terminals. The on-the-fly (OTF) variants omit controls at UC-enabled states and stop when all entries win. Direct-Full first materializes reachable buckets including those controls, then solves with the same UC-priority rule. Full+UC is an auxiliary full-construction variant that omits those controls during construction. At quiescence Lazy queries individual buckets, Eager all controls, and Update-first favors updates, testing O-DUCS’s priority [16] within our solver. Lazy favors stops/transfers, prioritizes states by endpoint mismatch and pending-command counts, and uses LIFO exploration before FIFO retries (Technical Appendix E); this order need not minimize exploration.

Design and measures. RQ3 uses one Windows/Xeon host, 64 GiB heap and 1,200 s JVM cap: 27×4 same-game cells plus 27 Legacy references. Only decisions receive four further trials; medi-ans/ranges require five completions. Measures separate exploration, policies, solver/checking time, total JVM time and peak working set. JVM adds frontend, startup, output and endpoints; Legacy uses its native timer. Mac pilots are excluded. Technical Appendices N and J.8 give full settings; interruptions/follow-ups remain recorded.

6.2 RQ1: Can Local Choices Change Feasibility?

Cell and Policy isolate component and requirement-lifetime obstructions (Section 2). These new cases exercise choices absent from the inherited DUCS-oriented benchmarks. We test whether merging those choices, with the remaining contract fixed, removes all completing policies; this is a mechanism question, not a frequency estimate over deployments.

What is merged. Partition transfers, old stops and new starts separately into blocks. A transfer block $B \subseteq I$ simultaneously permits all targets in $\prod_{i \in B} g_i(s_i)$; a boundary block removes or initializes its monitors at the same component tuple. Other active monitors compose member-label effects, checked to commute pairwise on every monitor/residual state. Mapped precedence must remain irreflexive and acyclic. Ordinary behavior, entries, endpoints, relations, initializers and targets stay fixed; quiescence and all member guards apply. These are comparison contracts, not DUCS encodings.

The following obstruction explains the Cell, Policy and PC2 merges directly; the rolling threshold then gives an exact quantitative boundary.

LEMMA 6.1 (MANDATORY-COMMAND OBSTRUCTION). *Let q_0 be an entry and B a nonempty set of commands pending there. Suppose a safe region R contains q_0 , retains all commands in B , and is closed under every safe successor whose label is outside B . If each command in B is disabled or has an unsafe outcome at every state in R , no winning policy covers q_0 .*

PROOF. Every goal exhausts pending commands, so a winner must execute a first command in B . Safety and closure keep its prefix in R , where that step is disabled or has an environment-chosen unsafe outcome. Avoiding it yields a non-goal deadlock or infinite run, also losing. $\square$

In Cell’s one-workpiece region, joint transfer is disabled. Policy retains its initial active bits until either grouped boundary violates the interval. PC2-Rolling’s enabled joint transfer violates availability (Section 6.2.1). These results concern specified merges, not native DUCS expressiveness.

Mechanisms and scope. Readiness/rejoining gates motivate staged replacement [27, 32]. Rolling begins with n ready replicas and no spares; its interval requirement keeps at least m ready. Each transfer boots its replicas; each has one UC readiness event and no waiting loop. Service self-loops are controllable. The target is all-new and ready.

PROPOSITION 6.2 (ROLLING THRESHOLD). *For $n \geq 2$, $1 \leq m < n$ and $1 \leq k \leq n$, fixed consecutive batches of size k (with a smaller final remainder) admit safe finite completion iff $k \leq n - m$.*

PROOF. Some mandatory batch has size k . Its transfer leaves at most $n - k$ ready replicas, so $k > n - m$ violates the interval. Otherwise, transfer one batch only when all replicas are ready, disable service, and retain every UC readiness outcome before the next transfer. Readiness never falls below $n - k \geq m$. With u old and b booting replicas, $2u + b$ strictly decreases on every selected transfer and every UC event, reaching zero exactly at the target. Thus all maximal runs finish without fairness. $\square$

The 15 Rolling separations compare $k = 1$ with $k = n$; the 70 threshold cells are separate. Canary adds health reports/recovery, DB-Rolling role-sensitive replacement, and Rolling+Audit overlapping duties. Policy isolates opposing boundaries; Threads supplies both-WIN/both-LOSS quiescence controls. Table 4 keeps all outcomes. Technical Appendix J.11 specifies mechanisms and assumption controls; their deployed prevalence is unestablished.

Answer. The suite yields 33/55 fine-WIN/merged-LOSS comparisons across five families. Granularity has no general refinement order: mixed-version UC loops can reverse the separation (Technical Appendix J.12). PC2 contributes no solver WIN/LOSS pair: its merged run times out. The

Table 4. Constructed granularity comparisons: each pair fixes one model and compares its fine and merged contracts. Policy merges boundaries; other rows merge transfers. n counts replicas/workers/stations. Canary bounds reported failures by $n - m$; otherwise m is an availability floor. The main total excludes Cell.

Family / control	Parameter domain	Pairs	WIN / LOSS	LOSS / LOSS	WIN / WIN	Inc.
Rolling	$n = 2 \dots 6, 1 \leq m < n$	15	15	0	0	0
Canary	$n = 2 \dots 6, 1 \leq m < n$	15	15	0	0	0
Policy	2 rule pairs	1	1	0	0	0
DB-Rolling	$n = 3, m = 2, 3$	2	1	1	0	0
Rolling+Audit	$n = 2, m = 1; 1$ audit pair	1	1	0	0	0
Threads (control)	$n = 2 \dots 6; \text{queue } 1, 2$	20	0	10	10	0
PC2-Rolling	2 arms; floor 1	1	0	0	0	1
Main total		55	33	11	10	1
Cell _n (reference)	$n = 2 \dots 10$	9	9	0	0	0

Completed columns read fine / merged; Inc. denotes unresolved. PC2's merged solver remains TO, not LOSS. Threads is a negative control; its queue capacities are 1 and 2, with both backpressure and saturated-offer regimes. Rolling+Audit counts transfer merging; boundary merging remains WIN and adds no pair here.

source argument below separately proves merged impossibility.

6.2.1 A Source-Derived Calibration Case. PC2-Rolling adds calibration before replacement arms resume production, requiring another arm operational. Calibration requests are controllable; completions are UC; transfers enter calibration states. Erasing calibration recovers the original operational process, without whole-problem equivalence. Fixed endpoints retain baseline requirements plus availability, giving 13 new starts. Endpoint availability tracks requests/completions; the interval also treats transfers as calibration entry. Transfers preserve initializer observers. The 25 boundaries and two transfers have no supplied order; Technical Appendix L compares generation responsibilities.

Generated update and certificate. The saved policy calibrates arm 1, waits for completion, then calibrates arm 2 and waits again. Transfers directly enter calibration, so no calibration request occurs. All 126 entries are covered; every outcome decreases rank from at most 37 and keeps 1

arm outside calibration. Link checks the full old–update–new composition. Table A6 and complete counts are in Technical Appendix L; this replay supplies no timing.

Source obligations through handover. Arm 1 retains $\Box(out_1 \Rightarrow Drilled_1 \wedge Cleaned_1 \wedge Painted_1)$. Successful tool results set its initially false fluents; reset clears them. At the eighth start on the saved path, no tool has succeeded. Initialization installs zero observers, so departure still requires all three tasks; falsely marking them finished would allow violating out_1 .

Every run from all 126 entries traverses the same thirteen-start chain. Each source invariant holds at its start and is preserved by the remaining starts until the endpoint takes over, proving all thirteen ACT obligations on this policy’s reached tuples. No ordinary event occurs on that suffix; GSM separately shows active ordinary behavior. This does not cover all initializer tuples or all 138 occurrences (Technical Appendix L.1).

Only 39/126 entries admit a safe full prefix; 87 fail (21 before entry, 66 during update). All admit an earlier violating cycle restoring zero observers. These limit history-wide interpretations, not ACT. Safe-history nonemptiness for 38,288 initializer tuples and runtime remain unchecked (Technical Appendix L).

PC2-Rolling: fixed-merge obstruction. All checked old entries admit both raw transfers. Before their mandatory joint transfer, old plants lack calibration actions; ordinary moves and boundaries preserve readiness even after releasing old control. Every joint outcome calibrates both arms and clears both interval readiness bits, reaching absorbing error. This source argument proves impossibility independently of solver completion. The merged solver and fine Direct-Full nevertheless time out at 7200 s/200 GiB; these are not losing decisions.

Inherited inputs and additional witnesses. On the nine inherited DUCS-oriented inputs, merging transfers, boundaries or both changes 0/54 Base/R1 decisions. This null result identifies a region where finer choices do not alter feasibility; it does not test the new independent-lifetime/readiness mechanisms. Their total raw transfers omit the explicit startup phase producing the Rolling obstruction; totality alone does not ensure a safe merge. States decrease in 44 comparisons and stay equal in 10; R2 names individual events and is excluded. These are feasibility comparisons, separate from RQ3’s performance results.

Additional restricted-transfer comparisons include both-LOSS and incomplete cases. Technical Appendix J gives aggregates and finite witnesses, with complete records in S1–S4; these populations are not pooled above.

6.3 RQ2: Do Policies Satisfy the Contract?

Winning and losing applications. Railcab has four component pairs, twelve requirement boundaries and a transfer with ENDOFTS/AC outcomes. Two new-AC states differ in approachingCrossing memory, so location alone determines neither initialization nor target compatibility. R1 adds seven interval guards; both exports pass certificate/Link checks. All 22 entries of each saved policy stop old requirements before the first new start; the intervening ordinary-event count is 1–4 for Base and zero for R1, whose interval guards remain active. These two qualitative exports do not characterize all benchmark gaps or suppressed events (Technical Appendix J.9; S4).

Industry/R1 forbids validation between old-stop and new-start and loses at 42/131 entries under independent exhaustive checking. At its trap, GF1/TOR validation violates new requirements; DSD1 permits an infinite rejection cycle or deadlock if disabled. The full losing closure excludes all orders. Native DUCS and its aligned fork instead win on the source model with a subsequent-control objective [28, Defs. 4.1–4.3]; the adapted problems differ, so this does not isolate endpoint

fixing. Exempting GF1 yields all 131 WIN through earlier paths, not trap escape. Technical Appendix J retains details; neither check supplies timing.

Answer. Lazy and Direct-Full match all expected results from the independent raw labeled transition systems (LTSs).

Independent reconstruction and certificates. Python independently constructs graphs and certificates for 46 fixtures: $46 \times 2 = 92$ jobs yield 36 WIN, 48 LOSS and 8 invalid (missing events, deadlocks or suppressed UC). All 84 eligible jobs pass certificate/exhaustive checks; all WIN jobs pass Link. Explicit successors are separate; frontend and endpoint/goal code are shared. These jobs are not independent applications.

Six monitor-free target/quiescence fixtures independently reconstruct restricted FSP, endpoints, complete games and goals in Python, bypassing Java successor/goal code. Across 27 Java exports, 138 NEW initializer tables match 1,587 reconstructed observer/monitor entries. This reconstruction uses Java’s compiled monitors; it does not validate their FSP compilation, physical-state decoding, requirement intent or general history interpretation. These are non-performance checks (Technical Appendix G).

Checking initializer interpretation. Table 5 distinguishes the 138 NEW and 135 UPD checks from the direct ACT arguments. Empty scopes pass vacuously; inclusion does not establish intent. Technical Appendix G gives the formula, label, reset and automaton checks and their complete counts.

Source obligation over ordinary actions. GSM changes $\square\text{-decode.3}$ to $\square\text{-decode.2}$. From every old entry, reach the first RECEIVED state; then

$$\text{RECEIVED} \xrightarrow{\text{stop,start}} \text{RECEIVED} \xrightarrow{\text{decode.3}} \text{DECODED} \xrightarrow{\text{out put}} \text{SENDER} \xrightarrow{\rho} \text{SENDER}_{\text{new}}.$$

The preceding *receive* sets the *decode.2* observer to zero; boundaries preserve it. The saved initializer maps zero to monitor state zero: *decode.2* enters error, all other labels preserve zero. Thus the new obligation constrains an actual ordinary choice. Transfer matches sender, observer and new-monitor zero to the endpoint. The pre-stop distance is at most seven; adding the five displayed events gives rank at most twelve. Consecutive stop/start satisfies R1’s gap ban. This source-derived analytical policy proves ACT, not a new solver result; a separate 45-pair source check establishes E initialization (Technical Appendix M).

The 25 WIN certificates have 8–1,071 update states and maximum ranks 4–34, consistent across five repeats. Linked old–update–new controllers have 15–1,260 states (S4). Ranks bound events, not downtime or optimal length (Technical Appendix J.8).

6.4 RQ3: Does Lazy Reduce Exploration?

Answer. Lazy avoids states and queries in all 14 completed Direct-Full pairs. Within the fixed budget it decides 26 contracts, Eager 17 and Direct-Full 14; all paired decisions agree (Table 6). Update-first is faster on 8/24 paired solver medians but adds two timeouts. Lazy returns LOSS for Industry/R1 and times out on PC2/R2 at 1,200 s/64 GiB (fixed), 3,600 s/64 GiB (sensitivity) and 7,200 s/200 GiB (S4.6, earlier series); the limiting resource is unknown.

Legacy yields 9 WIN, 13 OOM and 5 unmeasured snapshot-cap cases. Its differing subsequent-control objective precludes same-game ratios or impossibility inferences (Technical Appendix J).

Figure 2 plots all 14 completed pairs; Appendix D gives exact states, queries and costs. In R2, Update-first uses fewer states and a lower solver median on six of eight completed pairs, showing order sensitivity (Technical Appendix J).

Capped Direct-Full attempts give total-JVM lower bounds, not solver ratios or timing medians (Technical Appendix J.10).

Table 5. What the guarantees establish and where their evidence ends. The three layers are game correctness, requirement interpretation and execution.

Layer/scope	Established result	Evidence and remaining obligation
Game	Safety, all-outcome finite completion and compatible handover from all entries	Theorem 4.1; checking trusts supplied successor/endpoint predicates
A: safe history	NEW language included in every admitted safe-history residual	138 occurrences (46 requirements, three variants); nonempty scopes unverified beyond one PC2 tuple
E: from entry	UPD initialization equals the entry residual	135 occurrences: 46 R1/89 R2; guards/stage fluents justify entry values
ACT: from start	Source invariants hold from activation through handover and continuation	PC2: all 13 new requirements, all 126 entries; no active ordinary event. GSM: all entries with decode/output
Cell duty	Actual occupancy initializes inspection debt; the monitor exactly enforces its update rules	Bit/monitor induction in Section 5.1; both entries, including ordinary inspection
Execution	Observed runs follow policy edges and finish within entry rank	Conditional A1–A4; adapter conformance and real-time bounds unvalidated

H (whole history) is not claimed: only 39/126 PC2 entries admit a safe full prefix; 87 do not. Selecting a deployment’s requirements remains the modeler’s responsibility.

Table 6. All 27 contracts per method. TO denotes one capped trial, not a timing median. Ratios summarize heterogeneous completed pairs; values above one favor Lazy.

Method	WIN/LOSS/TO	Pairs	Faster than Lazy	Solver ratio	Total JVM ratio
Lazy	25/1/1	—	—	—	—
Eager	17/0/10	17	0	2.04 [1.18–4,575.05]	1.60 [1.00–144.85]
Update-first	23/1/3	24	8	1.07 [0.19–39.76]	1.00 [0.32–6.10]
Direct-Full	14/0/13 [†]	14	0	9.14 [1.58–2,123.81]	2.27 [1.00–130.66]

TO: timeout; no OOM/unmeasured cases. [†] Workflow/R2 retains its interruption and later same-setting timeout (S4).

Comparator/Lazy ratios are median [min–max] across completed pairs of five-run medians. Faster counts lower comparator solver time. JVM includes frontend, startup, endpoints and output; solver includes checking. Unresolved costs have no ratio.

Separating construction choices. Full+UC cannot isolate pruning time because builds, hosts and repetitions differ; no unpruned OTF is implemented (Technical Appendix J.10).

Scaling limits. All methods finish the independent family for $n = 2, \dots, 20$. At $n = 20$, Lazy and Update-first discover 21 states each, Eager 211, and Direct-Full 1,048,576 (S4).

Travel reaches timeouts and out-of-memory failures under all methods, including expanded-budget timeouts. These separate trials never enter medians; Technical Appendix J.10 retains all populations and adverse outcomes.

7 Related Work

Zhang and Cheng formalize temporal adaptation semantics [37]; scenario-based synthesis derives updates from history-based updatability criteria [12, 23]. Local dynamic updating addresses component replacement [22]. PROTEOS filters replacement interfaces [13, Secs. 4.3–4.6]; workflow

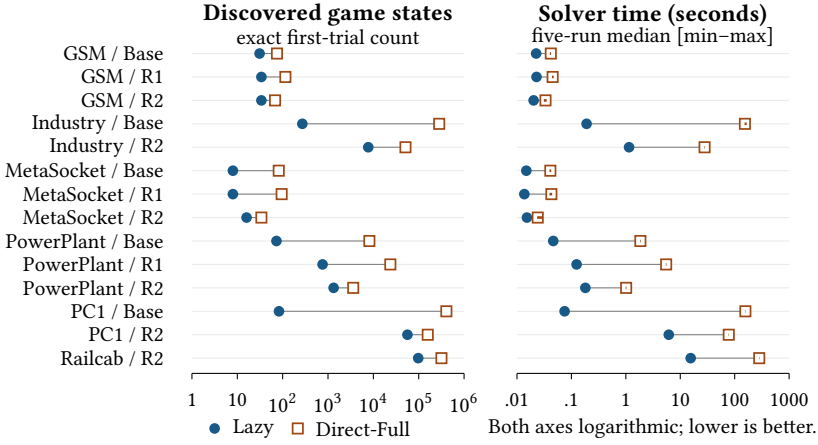


Fig. 2. All 14 completed WIN/WIN pairs. Solver time includes checking but excludes frontend, endpoint preparation and JVM startup. The other 13 Direct-Full contracts time out (Figure A4) and are omitted. This compares whole procedures.

updating blocks tasks [31, Secs. 4.3, 5.2]; Kitsune migrates C programs [14, Sec. 2]; network synthesis preserves packet-processing properties [24]. O-DUCS prioritizes updates [16]; Live Synthesis carries unfinished obligations [11]. RS builds on scopes [10] and residual rewriting [30]. State attraction bounds startup steps under weak invariance [29]. Compositional [26] and modular [36] control address specification structure. OTF-DCS requires marked extensions [8, Sec. 3]; FG requires finite quiescent handover on every maximal run. The objectives differ from FG-DUCS.

Thuijsman and Reniers [33, Secs. 4–7] support individual/simultaneous feature reconfiguration, feature-dependent requirements, and component-state retention or reset. Their nonblocking supervisor permits a marked continuation from every reachable state. FG-DUCS schedules transfers and independent requirement lifetimes with declared monitor initialization, requiring every maximal update run to finish at a supplied controller.

8 Threats to Validity

Perpetual UC activity can prevent completion. Decoding, history completeness and A1–A4 remain unvalidated; repair is excluded. Constructed cases establish mechanisms, not deployed prevalence, and selected conditions may favor them. They assume finite startup, reporting, recovery and election. Only 39/126 PC2 entries have a safe full prefix for the selected requirement.

Shared authorship, AI and code risk correlated faults; independent parsing covers restricted FSP. Direct-Full is in-house; Legacy’s objective differs. Pruning isolation, external solvers and randomized orders remain untested. Ratios omit unresolved costs; extended trials lack medians. Sampling can miss peaks; RSS identifies neither heap occupancy nor bottlenecks. S4 retains interruptions.

Null/LOSS results, faster Update-first, lower Eager/Update-first memory and Travel/PC limits bound gains. PC2’s merged timeout remains unresolved. Fixtures/repeats are not independent applications; timing uses one machine. Inherited inputs lack precedence; only Railcab transfers branch. Exponential separation assumes the default order.

Conclusion. FG-DUCS makes component changes and individual requirement lifetimes part of the update-synthesis problem. Its construction and lazy solver provide safe finite completion between supplied controllers, under declared requirement meanings and execution premises.

Data Availability. The anonymous repository [4], snapshot fgducs-c68-20261002, provides the MTSA-derived source, all 27 application contracts, scripts, generated results and split raw-output archive, together with the matching manuscript and complete build sources. The submission comprises this main paper and `supplementary_material.pdf`, which combines the technical appendix and experimental supplement with complete proofs, model definitions and result tables. Section labels distinguish Technical Appendix A–P from Supplement S1–S5; bookmarks and continuous page numbers support navigation. The claim-to-evidence map gives saved-data checks and build commands. The build produces the main paper and integrated supplement, retaining the two component PDFs for compatibility; saved evidence checks do not rerun performance experiments. Third-party dependencies are retrieved from official upstream sources.

References

- [1] Bowen Alpern and Fred B. Schneider. 1985. Defining Liveness. *Inform. Process. Lett.* 21, 4 (1985), 181–185. doi:10.1016/0020-0190(85)90056-0
- [2] Gal Amram, Shahar Maoz, Itai Segall, and Matan Yossef. 2022. Dynamic Update for Synthesized GR(1) Controllers. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. ACM, New York, NY, USA, 786–797. doi:10.1145/3510003.3510054
- [3] Jesper Andersson, Luciano Baresi, Nelly Bencomo, Rogério de Lemos, Alessandra Gorla, Paola Inverardi, and Thomas Vogel. 2013. Software Engineering Processes for Self-Adaptive Systems. In *Software Engineering for Self-Adaptive Systems II*, Rogério de Lemos, Holger Giese, Hausi A. Müller, and Mary Shaw (Eds.). Lecture Notes in Computer Science, Vol. 7475. Springer, Berlin, Germany, 51–75. doi:10.1007/978-3-642-35813-5_3
- [4] Anonymous FG-DUCS Authors. 2026. FG-DUCS Evaluation Artifact. Anonymous replication package. Manuscript and evidence snapshot, 2 October 2026; tag fgducs-c68-20261002. <https://github.com/research-artifact-archive/research-artifact/tree/fgducs-c68-20261002>
- [5] Franck Cassez, Alexandre David, Emmanuel Fleury, Kim G. Larsen, and Didier Lime. 2005. Efficient On-the-Fly Algorithms for the Analysis of Timed Games. In *CONCUR 2005 – Concurrency Theory (Lecture Notes in Computer Science, Vol. 3653)*. 66–80. doi:10.1007/11539452_9
- [6] Betty H. C. Cheng, Rogério de Lemos, Holger Giese, Paola Inverardi, Jeff Magee, Jesper Andersson, Basil Becker, Nelly Bencomo, Yuriy Brun, Bojan Cukic, Giovanna Di Marzo Serugendo, Schahram Dustdar, Anthony Finkelstein, Cristina Gacek, Kurt Geihs, Vincenzo Grassi, Gabor Karsai, Holger M. Kienle, Jeff Kramer, Marin Litoiu, Sam Malek, Raffaella Mirandola, Hausi A. Müller, Sooyong Park, Mary Shaw, Matthias Tichy, Massimo Tivoli, Danny Weyns, and Jon Whittle. 2009. Software Engineering for Self-Adaptive Systems: A Research Roadmap. In *Software Engineering for Self-Adaptive Systems*. Lecture Notes in Computer Science, Vol. 5525. Springer, Berlin, Germany, 1–26. doi:10.1007/978-3-642-02161-9_1
- [7] Alessandro Cimatti, Marco Roveri, and Paolo Traverso. 1998. Strong Planning in Non-Deterministic Domains via Model Checking. In *Proceedings of the Fourth International Conference on Artificial Intelligence Planning Systems (AIPS '98)*. AAAI Press, Menlo Park, CA, USA, 36–43. <https://cdn.aaai.org/AIPS/1998/AIPS98-005.pdf>
- [8] Daniel Ciolek, Matias Duran, Florencia Zanollo, Nicolás Pazos, Julián Braier, Víctor Braberman, Nicolás D'Ippolito, and Sebastián Uchitel. 2023. On-the-Fly Informed Search of Non-Blocking Directed Controllers. *Automatica* 147 (2023), 110731. doi:10.1016/j.automatica.2022.110731
- [9] Nicolas D'Ippolito, Dario Fischbein, Marsha Chechik, and Sebastian Uchitel. 2008. MTSA: The Modal Transition System Analyser. In *2008 23rd IEEE/ACM International Conference on Automated Software Engineering*. IEEE, Piscataway, NJ, USA, 475–476. doi:10.1109/ASE.2008.78
- [10] Matthew B. Dwyer, George S. Avrunin, and James C. Corbett. 1999. Patterns in Property Specifications for Finite-State Verification. In *Proceedings of the 21st International Conference on Software Engineering (ICSE '99)*. ACM, New York, NY, USA, 411–420. doi:10.1145/302405.302672
- [11] Bernd Finkbeiner, Felix Klein, and Niklas Metzger. 2022. Live Synthesis. *Innovations in Systems and Software Engineering* 18, 3 (2022), 443–454. doi:10.1007/s11334-022-00447-5
- [12] Carlo Ghezzi, Joel Greenyer, and Valerio Panzica La Manna. 2012. Synthesizing Dynamically Updating Controllers from Changes in Scenario-Based Specifications. In *Proceedings of the 7th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS '12)*. IEEE, Piscataway, NJ, USA, 145–154. doi:10.1109/SEAMS.2012.6224401
- [13] Cristiano Giuffrida, Anton Kuijsten, and Andrew S. Tanenbaum. 2013. Safe and Automatic Live Update for Operating Systems. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '13)*. ACM, New York, NY, USA, 279–292. doi:10.1145/2451116.2451147

- [14] Christopher M. Hayden, Karla Saur, Edward K. Smith, Michael Hicks, and Jeffrey S. Foster. 2014. Kitsune: Efficient, General-Purpose Dynamic Software Updating for C. *ACM Transactions on Programming Languages and Systems* 36, 4, Article 13 (Oct. 2014), 38 pages. doi:10.1145/2629460
- [15] Maurice P. Herlihy and Jeannette M. Wing. 1990. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Transactions on Programming Languages and Systems* 12, 3 (1990), 463–492. doi:10.1145/78969.78972
- [16] Takanori Hirano, Takuto Yamauchi, Kenji Tei, and Shinichi Honiden. 2024. Dynamic Update Controller Synthesis using on-the-fly Exploration. *IEICE Technical Report* 123, 443 (March 2024), 76–78. In Japanese; non-peer-reviewed technical report. <https://ken.ieice.org/ken/paper/20240314Vccd/>
- [17] Martin Hofmann, Christian Neukirchen, and Harald Rueß. 2016. Certification for μ -Calculus with Winning Strategies. In *Model Checking Software (Lecture Notes in Computer Science, Vol. 9641)*. Springer, Cham, Switzerland, 111–128. doi:10.1007/978-3-319-32582-8_8
- [18] Jeff Kramer and Jeff Magee. 1990. The Evolving Philosophers Problem: Dynamic Change Management. *IEEE Transactions on Software Engineering* 16, 11 (1990), 1293–1306. doi:10.1109/32.60317
- [19] Orna Kupferman and Moshe Y. Vardi. 2001. Model Checking of Safety Properties. *Formal Methods in System Design* 19, 3 (2001), 291–314. doi:10.1023/A:1011254632723
- [20] Claus Lewerentz and Thomas Lindner. 1995. Case Study “Production Cell”: A Comparative Study in Formal Specification and Verification. In *KORSO: Methods, Languages, and Tools for the Construction of Correct Software*, Manfred Broy and Stefan Jähnichen (Eds.). Lecture Notes in Computer Science, Vol. 1009. Springer, Berlin, Germany, 388–416. doi:10.1007/BFb0015473
- [21] Jeff Magee and Jeff Kramer. 2006. *Concurrency: State Models and Java Programs* (2 ed.). John Wiley & Sons, Chichester, UK. <https://www.doc.ic.ac.uk/~jnm/book/>
- [22] Valerio Panzica La Manna. 2012. Local Dynamic Update for Component-Based Distributed Systems. In *Proceedings of the 15th ACM SIGSOFT Symposium on Component Based Software Engineering (CBSE ’12)*. ACM, New York, NY, USA, 167–176. doi:10.1145/2304736.2304764
- [23] Valerio Panzica La Manna, Joel Greenyer, Carlo Ghezzi, and Christian Brenner. 2013. Formalizing Correctness Criteria of Dynamic Updates Derived from Specification Changes. In *Proceedings of the 8th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS ’13)*. IEEE, Piscataway, NJ, USA, 63–72. doi:10.1109/SEAMS.2013.6595493
- [24] Jedidiah McClurg, Hossein Hojjat, Pavol Černý, and Nate Foster. 2015. Efficient Synthesis of Network Updates. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI ’15)*. ACM, New York, NY, USA, 196–207. doi:10.1145/2737924.2737980
- [25] Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. 2011. Certifying Algorithms. *Computer Science Review* 5, 2 (2011), 119–161. doi:10.1016/j.cosrev.2010.09.009
- [26] Sahar Mohajerani, Robi Malik, and Martin Fabian. 2014. A Framework for Compositional Synthesis of Modular Nonblocking Supervisors. *IEEE Trans. Automat. Control* 59, 1 (2014), 150–162. doi:10.1109/TAC.2013.2283109
- [27] MongoDB. [n. d.]. Upgrade to MongoDB Enterprise (Replica Set). MongoDB Database Manual, version 7.0. Accessed 2026-09-29. <https://www.mongodb.com/docs/v7.0/tutorial/upgrade-to-enterprise-replica-set/>
- [28] Leandro Nahabedian, Victor Braberman, Nicolás D’Ippolito, Shinichi Honiden, Jeff Kramer, Kenji Tei, and Sebastián Uchitel. 2020. Dynamic update of discrete event controllers. *IEEE Transactions on Software Engineering* 46, 11 (2020), 1220–1240. doi:10.1109/TSE.2018.2876843
- [29] Anas Nooruldeen and Klaus Werner Schmidt. 2015. State Attraction Under Language Specification for the Reconfiguration of Discrete Event Systems. *IEEE Trans. Automat. Control* 60, 6 (2015), 1630–1634. doi:10.1109/TAC.2014.2358811
- [30] Grigore Roşu and Klaus Havelund. 2005. Rewriting-Based Techniques for Runtime Verification. *Automated Software Engineering* 12, 2 (2005), 151–197. doi:10.1007/s10515-005-6205-y
- [31] Daniel Sokolowski, Pascal Weisenburger, and Guido Salvaneschi. 2022. Change Is the Only Constant: Dynamic Updates for Workflows. In *Proceedings of the 44th International Conference on Software Engineering (ICSE ’22)*. ACM, New York, NY, USA, 350–362. doi:10.1145/3510003.3510065
- [32] The Kubernetes Authors. [n. d.]. Deployments. Kubernetes Documentation. Accessed 2026-09-29. <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- [33] Sander B. Thuijsman and Michel A. Reniers. 2024. Supervisory Control for Dynamic Feature Configuration in Product Lines. *ACM Transactions on Embedded Computing Systems* 23, 5, Article 71 (2024), 25 pages. doi:10.1145/3579644
- [34] Stavros Tripakis and Karine Altisen. 1999. On-the-Fly Controller Synthesis for Discrete and Dense-Time Systems. In *Formal Methods (Lecture Notes in Computer Science, Vol. 1708)*. Springer, Berlin, Germany, 233–252. doi:10.1007/3-540-48119-2_15
- [35] Yves Vandewoude, Peter Ebraert, Yolande Berbers, and Theo D’Hondt. 2007. Tranquility: A Low Disruptive Alternative to Quiescence for Ensuring Safe Dynamic Updates. *IEEE Transactions on Software Engineering* 33, 12 (2007), 856–868. doi:10.1109/TSE.2007.70733

- [36] W. Murray Wonham and Peter J. Ramadge. 1988. Modular Supervisory Control of Discrete-Event Systems. *Mathematics of Control, Signals, and Systems* 1, 1 (1988), 13–30. doi:10.1007/BF02551233
- [37] Ji Zhang and Betty H. C. Cheng. 2006. Using Temporal Logic to Specify Adaptive Program Semantics. *Journal of Systems and Software* 79, 10 (2006), 1361–1369. doi:10.1016/j.jss.2006.02.062